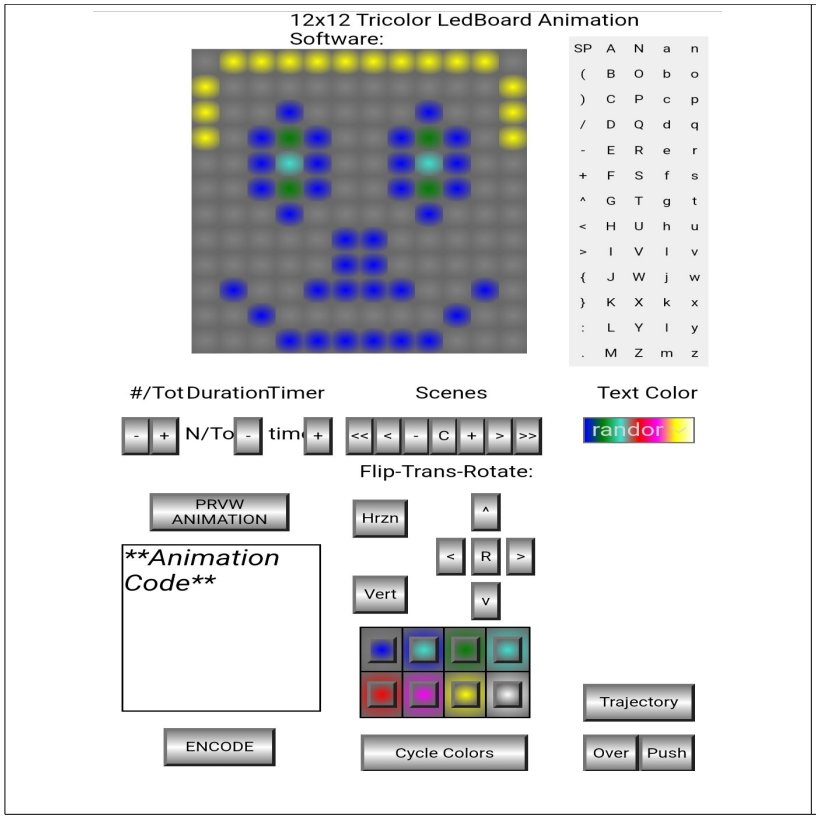

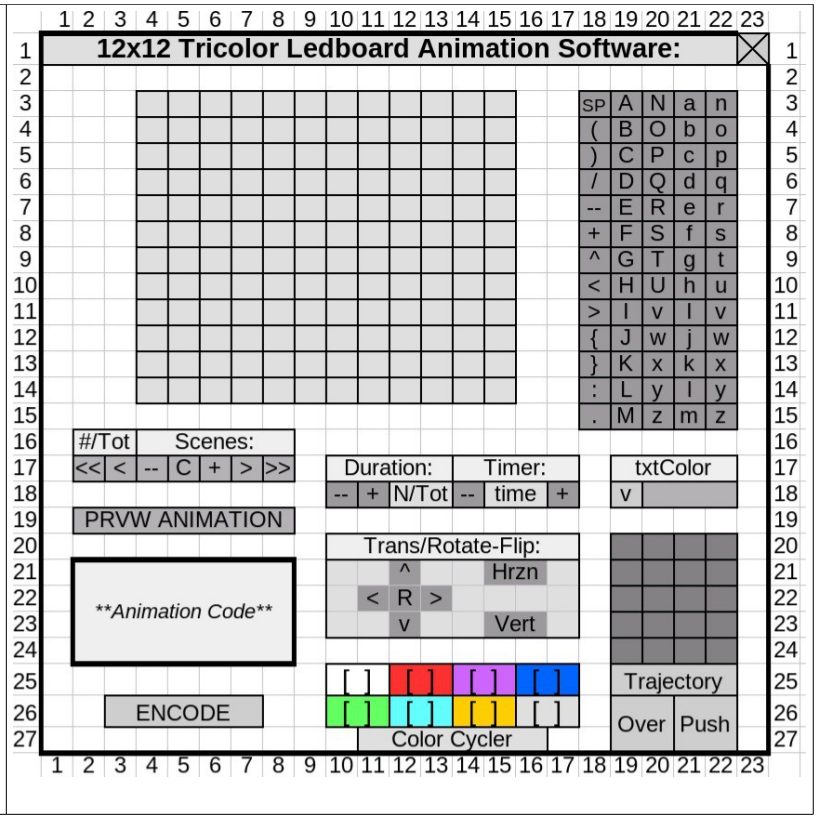


```
<!DOCTYPE html>
<html>
<head>
    <style>
   /*** Variables ***/
        :root {
          --Dw: calc((1/21.5)*100%);
          --Dh: calc((1/30)*100%);
              }


   /*** Main Grid ***/
        .grid-container {position: relative; width: 100%; height: 100vh; font-family: Arial, sans-
serif;}


   /*** Labels ***/
        .section-label {position: absolute; font-size: 75%;}



   /*** Buttons ***/
     /* LED Buttons */
        .btn-led {position: absolute; width: var(--Dw); height: var(--Dh);
            background-image: radial-gradient(circle at center, gray, dimgray 85%);
            cursor: pointer;  transition: background-color 0.2s; border: none; outline: none;}

     /* Character Buttons */
        .btn-char {position: absolute; width: var(--Dw); height: var(--Dh);
            background-color: light-gray; border: 0px solid #666; font-size: 5%;
            display: flex; align-items: center; justify-content: center;}
```

```
    /* Control Buttons */
       .btn-control {position: absolute; display: flex;
         width: var(--Dw); height: var(--Dh);
         background-image: linear-gradient(to top, gray, white, gray);
         font-size: 60%; align-items: center; justify-content: center;}

    /* Color Cycle Buttons */
       .color-cycle-btn {position: absolute; width: 70%; height: 60%; cursor: pointer;
       background-image: radial-gradient(circle at center, transparent, dimgray 85%);}


   /** Button Handlers **/
       .btn-led:hover, .btn-char:hover, .btn-control:hover,
       .btn-color:hover {filter: brightness(90%);}



  /*** Other Controls ***/
    /* Color Boxes */
       .color-box {position: absolute; width: calc(1.5*var(--Dw)); height: calc(1.75*var(--Dh));
         border: 1px solid black; display: flex; align-items: center; justify-content: center;
        background-image: radial-gradient(circle at center, transparent, dimgray);}

    /* Animation Code Box */
       .animation-code {position: absolute; border: 1px solid black;
           font-style: italic; font-size: 0.05em
           display: flex; align-items: center;
           justify-content: center; background-color: white;}

    /* Colored Text Box */
       .colored-text {position: absolute; width: calc(4*var(--Dw));
         align-items: center; justify-content: center; color: transparent;
         background-image: linear-gradient(to top, lightgray, transparent, lightgray);
         background-image: linear-gradient(to right, lightgray, transparent, lightgray);}

       .random-colored-text {position: absolute; width: calc(4*var(--Dw));
         align-items: center; justify-content: center; color: lightgray;
         background-image: linear-gradient(to top, lightgray, transparent, gray);
         background-image: linear-gradient(to right, blue, green, turquoise, red, magenta, yellow,
white);}

   </style>
</head>


<body>
  <div id="gridContainer" class="grid-container"></div>

  <script>

  //****Constants****//

   //**Color Pallet**//
       const colors = ['gray', 'blue', 'green', 'turquoise',
                       'red', 'magenta', 'yellow', 'white', 'random'];

   //**Character Grid**//
       const charGrid = [
```

```
            ['SP', 'A', 'N', 'a', 'n'],
            ['(', 'B', 'O', 'b', 'o'],
            [')', 'C', 'P', 'c', 'p'],
            ['/', 'D', 'Q', 'd', 'q'],
            ['-', 'E', 'R', 'e', 'r'],
            ['+', 'F', 'S', 'f', 's'],
            ['^', 'G', 'T', 'g', 't'],
            ['<', 'H', 'U', 'h', 'u'],
            ['>', 'I', 'V', 'l', 'v'],
            ['{', 'J', 'W', 'j', 'w'],
            ['}', 'K', 'X', 'k', 'x'],
            [':', 'L', 'Y', 'l', 'y'],
            ['.', 'M', 'Z', 'm', 'z']];

//*****Start Create Interface Function*****//
       function createInterface() {
            const container = document.getElementById('gridContainer');

      //****TriColor LED grid (12x12)****//
            for (let row = 0; row < 12; row++) {
                for (let col = 0; col < 12; col++) {
                    const button = document.createElement('button');
                    button.className = 'btn-led';
                    button.style.left = `calc(${(col + 3)}*var(--Dw))`;
                    button.style.top = `calc(${(row + 1.5)}*var(--Dh))`;

                    // Add color cycling functionality
                    button.currentColorIndex = 0;
                    button.addEventListener('click', function() {
                        // Increment color index, wrapping around to 0 when it reaches the end
                        this.currentColorIndex = (this.currentColorIndex + 1) % (colors.length-1);

                        // Set button background with gradient using the current color
                        const currentColor = colors[this.currentColorIndex];
                        this.style.backgroundImage = `radial-gradient(circle at center, $
{currentColor}, dimgray 85%)`;
                    });

                    container.appendChild(button);
                }}

      //****Alpha Character Grid****//
            for (let row = 0; row < charGrid.length; row++) {
                for (let col = 0; col < charGrid[row].length; col++) {
                    const button = document.createElement('button');
                    button.className = 'btn-char';
                    button.textContent = charGrid[row][col];
                    button.style.left = `calc(${(col + 16.5)}*var(--Dw))`;
                    button.style.top = `calc(${(row + 1)}*var(--Dh))`;
                    button.onclick = () => console.log(`Char ${charGrid[row][col]} clicked`);
                    container.appendChild(button);}}


      //****Labels****//
            const control_labels = [

            //Title//
              {textContent: '12x12 Tricolor LedBoard Animation Software:', x: 6.5, y: 0},
```

```
      //Static Labels//
        {textContent: 'Scenes', x: 11, y: 14.7},
        {textContent: 'Flip-Trans-Rotate:', x: 9, y: 17.8},
        {textContent: 'Duration', x: 2.8, y: 14.7},
        {textContent: 'Timer', x: 5.7, y: 14.7},
        {textContent: 'Text Color', x: 17.5, y: 14.7},

      //Dynamic Labels//
        {textContent: '#/Tot', x: 0.75, y: 14.7},
        {textContent: 'N/Tot', x: 2.75, y: 16.3},
        {textContent: 'time', x: 5.75, y: 16.3}];

       control_labels.forEach(lbl_ctrl => {
          const label = document.createElement('div');
          label.className = 'section-label';
          label.textContent = lbl_ctrl.textContent;
          label.style.left = `calc(${lbl_ctrl.x}*var(--Dw))`;
          label.style.top = `calc(${lbl_ctrl.y}*var(--Dh))`;
          container.appendChild(label);})


//****Buttons****//
      const control_buttons = [

        //Scene Controls//
          {text: '<<', x: 8.5, y: 16, w: 1},
          {text: '<', x: 9.5, y: 16, w: 1},
          {text: '-', x: 10.5, y: 16, w: 1},
          {text: 'C', x: 11.5, y: 16, w: 1},
          {text: '+', x: 12.5, y: 16, w: 1},
          {text: '>', x: 13.5, y: 16, w: 1},
          {text: '>>', x: 14.5, y: 16, w: 1},

        //Frame Controller//
          {text: '-', x: 0.5, y: 16, w: 1},
          {text: '+', x: 1.5, y: 16, w: 1},

        //Timer Control//
          {text: '-', x: 4.5, y: 16, w: 1},
          {text: '+', x: 7, y: 16, w: 1},

        //Direction-Rotation-Flip Panel//
          {text: '^', x: 13, y: 19, w: 1},
          {text: '<', x: 11.75, y: 20.75, w: 1},
          {text: '>', x: 14.25, y: 20.75, w: 1},
          {text: 'v', x: 13, y: 22.5, w: 1},
          {text: 'R', x: 13, y: 20.75, w: 1},
          {text: 'Hrzn', x: 8.75, y: 19.25, w: 2},
          {text: 'Vert', x: 8.75, y: 22.25, w: 2},

        //Color Cycler//
          {text: 'Cycle Colors', x: 9.05, y: 28.5, w: 6},

        //Alpha Motion//
          {text: 'Trajectory', x: 17, y: 26.5, w: 4},
          {text: 'Over', x: 17, y: 28.5, w: 2},
          {text: 'Push', x: 19, y: 28.5, w: 2},

        //Preview-Encode Animation//
```

```
            {text: 'PRVW ANIMATION', x: 1.5, y: 19, w: 5},
            {text: 'ENCODE', x: 2, y: 28.25, w: 4}];

         control_buttons.forEach(btn_ctrl => {
            const button = document.createElement('button');
            button.className = 'btn-control';
            button.textContent = btn_ctrl.text;
            button.style.left = `calc(${btn_ctrl.x}*var(--Dw))`;
            button.style.top = `calc(${btn_ctrl.y}*var(--Dh))`;
            button.style.width = `calc(${btn_ctrl.w}*var(--Dw))`;
            button.style.height = `calc(1.5*var(--Dh))`;
            container.appendChild(button);})



//****Color Boxes with Buttons****//
         const colorBoxControls = [
         {color: 'gray', x: 9, y: 24.25},
         {color: 'blue', x: 10.5, y: 24.25},
         {color: 'green', x: 12, y: 24.25},
         {color: 'turquoise', x: 13.5, y: 24.25},
         {color: 'red', x: 9, y: 26},
         {color: 'magenta', x: 10.5, y: 26},
         {color: 'yellow', x: 12, y: 26},
         {color: 'white', x: 13.5, y: 26}];

        //Append Color Boxes with Buttons to Container//
         colorBoxControls.forEach(ctrl => {
         const colorBox = document.createElement('div');
         colorBox.className = 'color-box';
         colorBox.style.backgroundColor = ctrl.color;
         colorBox.style.left = `calc(${ctrl.x}*var(--Dw))`;
         colorBox.style.top = `calc(${ctrl.y}*var(--Dh))`;
         const button = document.createElement('button');
         button.className = 'color-cycle-btn';
         button.style.backgroundColor = ctrl.color;

         colorBox.appendChild(button); container.appendChild(colorBox);});




//****Animation Code Box****//
      const codeBox = document.createElement('div');
      codeBox.className = 'animation-code';
      codeBox.textContent = '**Animation Code**';
      codeBox.style.left = `calc(0.5*var(--Dw))`;
      codeBox.style.top = `calc(21*var(--Dh))`;
      codeBox.style.width = `calc(7*var(--Dw))`;
      codeBox.style.height = `calc(6.5*var(--Dh))`;
      container.appendChild(codeBox);


//****Colored Text Box****//
      const comboBox = document.createElement('select');
      comboBox.id = 'myComboBox'; comboBox.className = 'colored-text';
      colors.forEach(color => {const option = document.createElement('option');
      option.value = color; option.textContent = color; comboBox.appendChild(option);});
```

```
      comboBox.addEventListener('change', function() {this.className = 'colored-text';
      if (this.selectedIndex === 8) {
      this.className = 'random-colored-text'; this.style.backgroundColor = '';
      } else {this.style.backgroundColor = colors[this.selectedIndex];}});

      comboBox.style.backgroundColor = colors[comboBox.selectedIndex];
      if (comboBox.selectedIndex === 3) {comboBox.className = 'random-colored-text';}
      comboBox.style.backgroundColor = colors[comboBox.selectedIndex];
      comboBox.style.left = `calc(17*var(--Dw))`; comboBox.style.top = `calc(16*var(--Dh))`;
      comboBox.style.cursor = 'pointer'; container.appendChild(comboBox);



  //****Color Cycler Function****//
      document.querySelectorAll('.color-cycle-btn').forEach((btn) => {
          btn.addEventListener('click', (e) => {
              e.stopPropagation();
              const currentBox = btn.parentElement;
              const currentColor = btn.style.backgroundColor;

           //*Find Color Index*//
              const currentIndex = colors.findIndex(color => {

               //*Convert Colors to RGB*//
                  const tempDiv = document.createElement('div');
                  tempDiv.style.color = color;
                  document.body.appendChild(tempDiv);
                  const rgbColor = window.getComputedStyle(tempDiv).color;
                  document.body.removeChild(tempDiv);

                  tempDiv.style.color = currentColor;
                  document.body.appendChild(tempDiv);
                  const rgbCurrent = window.getComputedStyle(tempDiv).color;
                  document.body.removeChild(tempDiv);

                  return rgbColor === rgbCurrent;});

           //*Get Next Color (cycle back to start if at end)*//
              const nextColorIndex = (currentIndex + 1) % (colors.length-1);
              const nextColor = colors[nextColorIndex];
          //**Update Button Colors**//
              btn.style.backgroundColor = nextColor;});});



            } //*****End Of CreateInterface Function*****//
//*****Create Interface On Page Load*****//
       createInterface();
    </script>
</body>
</html>
```